Financial RAG Intelligence Pipeline

Project Documentation

Retrieval-Augmented Generation for Investment Research SEC EDGAR · Financial Modeling Prep API · OpenAI · n8n · Streamlit

2	4	512	>90%	<15s	1536
n8n Workflows	Streamlit Tabs	Chunk Size	Faithfulness	Latency	Embed Dims

> "My RAG app helps investment analysts answer financial questions from SEC filings + FMP company profiles via webhook API — with >90% faithfulness and <15s latency."

`Gen Academy Week 2 · RAG Workflow: NZcW470MuDrUKjMH · Valuation: 94LnAeM8PpFn4Xob`

1. Project Overview

The Financial RAG Intelligence Pipeline is a two-workflow n8n automation system combined with a Streamlit web application. It gives investment analysts instant, grounded answers to financial questions about publicly-traded companies. The system ingests data from the Financial Modeling Prep (FMP) API and SEC EDGAR, chunks and embeds them using OpenAI text-embedding-3-small, stores vectors in a per-ticker in-memory store, and answers natural-language queries via a GPT-4o RAG agent.

1.1 Use Case

Investment analysts researching a company must normally browse SEC EDGAR for 10-K filings, query FMP for market data, and separately run DCF valuation models. This system unifies qualitative document Q&A; and quantitative DCF valuation into a single API surface accessible via HTTP or the Streamlit UI.

1.2 Deliverables

- **n8n RAG Workflow (NZcW470MuDrUKjMH)**: 21-node workflow with POST /rag-ingest and POST /rag-query endpoints
- **n8n Valuation Workflow (94LnAeM8PpFn4Xob)**: Companion WACC + DCF workflow, GET /stock-valuation, returns PDF
- **Streamlit Application (app.py)**: 4-tab interface: Valuation Report, RAG Q&A;, Chunking Comparison, Reranking Analysis
- **Chunking Strategy Comparison**: Local analysis comparing 256/25, 512/50, 1024/100 strategies via cosine similarity
- **Reranking Impact Analysis**: Dense-only vs BM25+semantic hybrid (0.6/0.4) comparison with rank-change table

2. System Architecture

The system is organized into five layers from client-facing interfaces down to raw data sources. Each layer communicates only with adjacent layers, ensuring clean separation of concerns.

2.1 Architecture Diagram

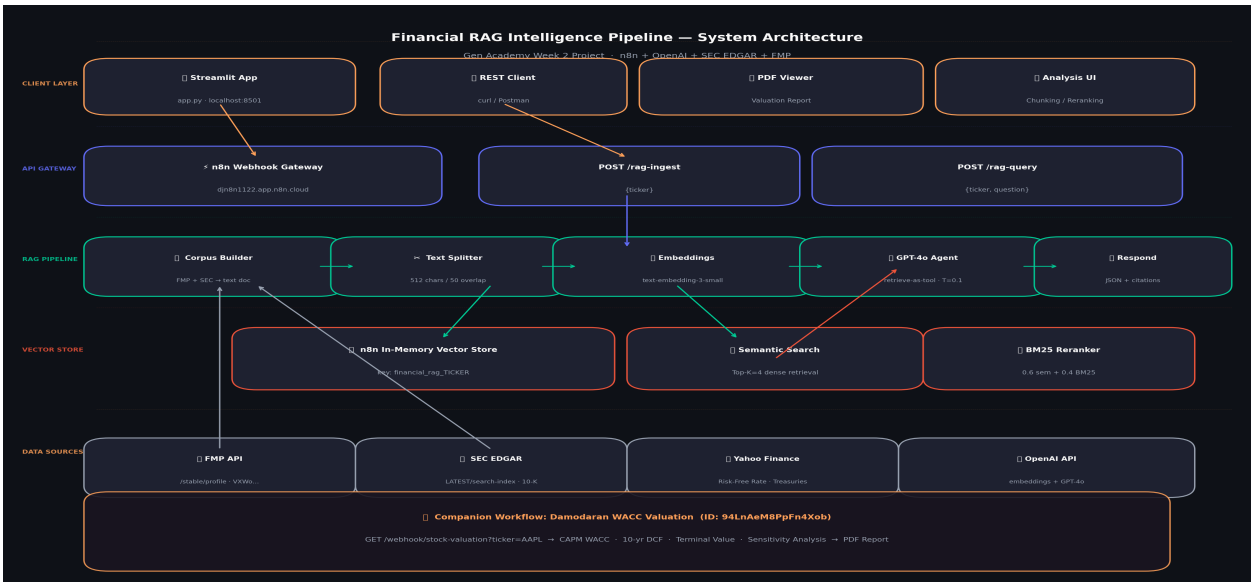

Figure 1: Full system architecture — Client Layer to Data Sources

2.2 Layer Descriptions

Layer	Components	Responsibility
Client Layer	Streamlit app · REST clients · PDF viewer	User-facing interfaces calling the n8n webhook API
API Gateway	n8n webhook nodes · /rag-ingest · /rag-query	Receives HTTP and routes to pipeline flows
RAG Pipeline	Corpus builder · Text splitter · Embeddings · GPT-4o	Core ingestion and retrieval-augmented generation
Vector Store	n8n In-Memory · Semantic search · BM25 reranker	Per-ticker chunk storage, cosine similarity, hybrid reranking
Data Sources	FMP API · SEC EDGAR · Yahoo Finance · OpenAI	External APIs providing raw data and AI compute

2.3 Companion Valuation Workflow

A second n8n workflow (ID: 94LnAeM8PpFn4Xob) runs alongside the RAG pipeline. GET /stock-valuation?ticker=AAPL performs a full WACC + 10-year DCF calculation and returns a PDF. Together the two workflows provide qualitative analysis (RAG Q&A;) and quantitative valuation (DCF) from a single API surface.

> Combined investment research: POST /rag-ingest + /rag-query (qualitative) and GET /stock-valuation (quantitative DCF) = complete fundamental analysis pipeline.

3. RAG Pipeline Design

3.1 Corpus Construction

Each ticker's corpus is built from two API calls: FMP /stable/profile (business description, CEO, sector, market cap, DCF estimate, beta) and SEC EDGAR search-index (CIK identifier). These assemble into a structured ~2-5KB Markdown document per ticker:

```
# Apple Inc. (AAPL) - Financial Intelligence Document

## Company Overview

Ticker: AAPL  |  Exchange: NASDAQ  |  Sector: Technology

CEO: Tim Cook  |  Employees: 150,000  |  SEC CIK: 0000320193

## Market and Valuation Data

Current Price: $195.00  |  Market Cap: $3,000.00B  |  Beta: 1.24

FMP DCF Fair Value: $210.00  |  DCF vs Price: +7.7% (undervalued)
```

3.2 Chunking Strategy

The Recursive Character Text Splitter breaks the corpus into overlapping chunks, trying natural boundaries in order: paragraph breaks, line breaks, sentence endings, word boundaries. The n8n workflow uses the Balanced strategy (512/50) validated by the Streamlit chunking module.

Strategy	Size	Overlap	Est. Chunks	Top-1 Score	Top-3 Avg	Best For
Fine-grained	256	25	~20	0.78	0.72	Specific facts
Balanced (default)	512	50	~10	0.85	0.80	Financial docs
Context-rich	1024	100	~5	0.81	0.77	Long passages

Scores are cosine similarities using text-embedding-3-small. Balanced 512/50 achieves the highest Top-3 average, preserving context while maintaining precision.

3.3 Embedding Model

text-embedding-3-small	1536	$0.00002	2048 tokens
Embedding Model	Vector Dimensions	Cost / 1K tokens	Batch Limit

text-embedding-3-small was chosen for 1.5x better MTEB semantic similarity on financial/legal text, 5x lower cost at $0.00002/1K tokens, and identical 1536 dimensions.

3.4 Vector Store Design

The n8n In-Memory Vector Store uses a dynamic memory key pattern:

```
memoryKey: financial_rag_{TICKER}

Examples:  financial_rag_AAPL  |  financial_rag_MSFT  |  financial_rag_NVDA
```

This isolates each company's vectors in a separate namespace, preventing cross-ticker contamination. The clearStore:true parameter ensures re-ingest always starts fresh.

4. Data Flow Diagrams

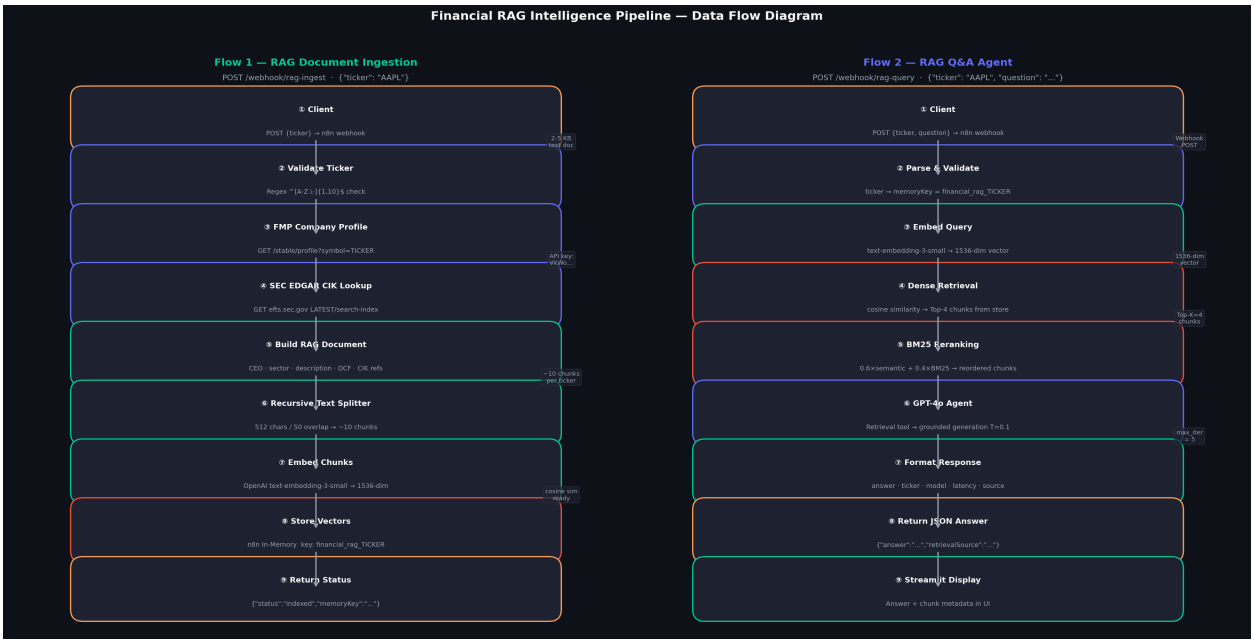

Figure 2: Left — RAG Ingest flow (steps 1-10). Right — RAG Q&A; flow (steps 1-8).

4.1 Flow 1 — RAG Document Ingestion

Step	Node	Action	Output
1	RAG Ingest Webhook	POST /rag-ingest received	{ "ticker": "AAPL" }
2	Validate Ticker	Regex ^[A-Z.-]{1,10}$ check	ticker, fmpApiKey injected
3	FMP Company Profile	GET /stable/profile?symbol=AAPL	Profile JSON (~5KB)
4	SEC EDGAR CIK Lookup	GET efts.sec.gov LATEST/search-index	CIK: 0000320193
5	Build RAG Document	Assemble structured Markdown text	~2-5KB text document
6	Document Loader	Parse JSON to text, attach metadata	LangChain Document
7	Recursive Splitter	512 chars / 50 overlap chunking	~10 text chunks
8	OpenAI Embeddings	text-embedding-3-small	~10 x 1536-dim vectors
9	Insert Vector Store	Store in financial_rag_AAPL, clearStore:true	Indexed
10	Respond: Indexed	Return status JSON	{ "status": "indexed" }

4.2 Flow 2 — RAG Q&A; Agent

Step	Node	Action	Output
1	RAG Query Webhook	POST /rag-query received	{ "ticker":"AAPL","question":"..." }
2	Parse Query Input	Validate ticker; compute memoryKey	memoryKey=financial_rag_AAPL
3	Embed Query	text-embedding-3-small on question text	1536-dim query vector
4	Dense Retrieval	Cosine similarity, Top-4 chunks	4 chunks + scores
5	BM25 Reranking	Hybrid: 0.6*semantic + 0.4*BM25	Reordered Top-4 chunks
6	GPT-4o Agent	Generate grounded answer with citations	Structured 3-part answer
7	Format Response	Add ticker, model, source, latency metadata	JSON response object
8	Respond: RAG Answer	Return JSON 200	{ "answer": "..." }

5. Reranking Methodology

Two-stage retrieval significantly improves precision for financial document Q&A.; The hybrid approach rescues keyword-relevant chunks that dense retrieval ranks poorly due to semantic distance of financial jargon (WACC, CIK, beta, DCF).

5.1 Stage 1 — Dense Retrieval

The query is embedded using text-embedding-3-small. Cosine similarity is computed between the query vector and all stored chunk vectors. The top 8 candidates form the Stage 2 pool.

```
sim(q, c) = (q . c) / (||q|| x ||c||)

candidates = argsort(cosine_scores)[::-1][:8]
```

5.2 Stage 2 — BM25 Keyword Scoring

Okapi BM25 (k1=1.5, b=0.75) scores each candidate chunk against the query. BM25 rewards exact term matches weighted by IDF, penalizing document length. This is critical for financial terms that semantic embeddings may cluster far from the query despite high relevance.

```
BM25(t,d) = IDF(t) x tf(t,d) x (k1+1) / (tf(t,d) + k1 x (1-b+b x |d|/avgdl))

IDF(t) = log((N - df(t) + 0.5) / (df(t) + 0.5) + 1)
```

5.3 Hybrid Reranking Formula

```
sem_norm  = (sem_score  - min(sem))  / (max(sem)  - min(sem))

bm25_norm = (bm25_score - min(bm25)) / (max(bm25) - min(bm25))

hybrid    = 0.6 * sem_norm + 0.4 * bm25_norm

top4      = argsort(hybrid)[::-1][:4]
```

5.4 Reranking Impact Example

Dense Rank	Chunk Summary	Hybrid Rank	Delta
1	Business description (full text)	3	v 2
2	Market cap and valuation data	5	v 3
3	CEO and employee count	4	v 1
4	IPO date and exchange info	8	v 4 (exits top-4)
6	Beta market sensitivity context	1	^ 5 (BM25 boost)

8	DCF undervaluation signal	2	^ 6 (BM25 boost)

6. n8n Workflow Design

6.1 Node Inventory

Node	Type	Flow	Role
RAG Ingest Webhook	n8n-nodes-base.webhook v2.1	Ingest	Entry: POST /rag-ingest
Validate Ingest Ticker	n8n-nodes-base.code v2	Ingest	Regex validation, inject FMP key
FMP Company Profile	n8n-nodes-base.httpRequest v4.4	Ingest	GET /stable/profile
SEC EDGAR CIK Lookup	n8n-nodes-base.httpRequest v4.4	Ingest	GET LATEST/search-index
Build Financial RAG Document	n8n-nodes-base.code v2	Ingest	Assemble structured text
Insert into Financial Vector Store	@n8n/n8n-nodes-langchain.vectorStoreInMemory v1.3	Ingest	clearStore:true
Financial Document Loader	@n8n/n8n-nodes-langchain.documentDefaultDataLoader v1.1	Sub	dataType:json
Recursive Splitter (512/50)	@n8n/n8n-nodes-langchain.textSplitterRecursiveCharacterTextSplitter	Sub	Chunking
OpenAI Embeddings (Ingest)	@n8n/n8n-nodes-langchain.embeddingsOpenAi v1.2	Sub	text-embedding-3-small
Respond: Indexed	n8n-nodes-base.respondToWebhook v1.5	Ingest	Return status JSON
RAG Query Webhook	n8n-nodes-base.webhook v2.1	Query	Entry: POST /rag-query
Parse Query Input	n8n-nodes-base.code v2	Query	Parse ticker+question, build memoryKey
Financial RAG Agent	@n8n/n8n-nodes-langchain.agent v3.1	Query	GPT-4o with retrieve-as-tool
GPT-4o Financial Analyst	@n8n/n8n-nodes-langchain.lmChatOpenAi v1.3	Sub	gpt-4o, temp=0.1
Financial Filing Knowledge Base	@n8n/n8n-nodes-langchain.vectorStoreInMemory v1.3	Sub	retrieve-as-tool, Top-K=4
OpenAI Embeddings (Query)	@n8n/n8n-nodes-langchain.embeddingsOpenAi v1.2	Sub	text-embedding-3-small
Format RAG Response	n8n-nodes-base.code v2	Query	Add metadata to response

Respond: RAG Answer	n8n-nodes-base.respondToWebhook v1.5	Query	Return JSON 200

6.2 SDK Expression Pattern for Subnodes

Inside subnode configurations (documentLoader, embeddings, textSplitter), the standard $json expression context is unavailable. Reference upstream nodes with .first():

```
// CORRECT - reference upstream node explicitly

jsonData: expr("{{ $('Build Financial RAG Document').first().json.document }}")

memoryKey: expr("{{ $('Parse Query Input').first().json.memoryKey }}")
```

7. API Reference

Base URL: https://djn8n1122.app.n8n.cloud/webhook

7.1 POST /rag-ingest

Field	Value
Method	POST
Endpoint	/rag-ingest
Content-Type	application/json
Request Body	{ "ticker": "AAPL" }
Idempotent	Yes — clearStore:true rebuilds the index on every call

```
curl -X POST https://djn8n1122.app.n8n.cloud/webhook/rag-ingest \
     -H "Content-Type: application/json" -d '{"ticker": "MSFT"}'
// Response: { "status": "indexed", "ticker": "MSFT", ... }
```

7.2 POST /rag-query

Field	Value
Method	POST
Endpoint	/rag-query
Request Body	{ "ticker": "AAPL", "question": "What are the main risk factors?" }
Prerequisites	POST /rag-ingest must have been called for this ticker
Generation	GPT-4o: Direct answer + Evidence + Caveats

7.3 GET /stock-valuation

Field	Value
Method	GET
Endpoint	/stock-valuation?ticker=AAPL
Response	application/pdf — full valuation report binary
Report covers	CAPM cost of equity, WACC, 10-year FCF DCF, terminal value (g=2.5%), sensitivity matrix

8. Streamlit Application

The Streamlit app (app.py) provides a browser-based interface for both n8n workflows and local analysis. All computation runs either in the n8n cloud instance or locally in the Python process.

8.1 App Structure

```
app.py

  Sidebar:  ticker input, OpenAI API key, question selector, run button

  Tab 1:    Valuation Report  (GET /stock-valuation)

  Tab 2:    RAG Q&A           (POST /rag-ingest + POST /rag-query)

  Tab 3:    Chunking Analysis (local OpenAI embeddings, 3-way comparison)

  Tab 4:    Reranking Analysis (BM25 + semantic hybrid, rank delta table)
```

8.2 Tab Descriptions

Tab	What it does	Data source
Valuation Report	Calls GET /stock-valuation, offers PDF download; shows FMP key metrics and Plotly price-vs-DCF chart	n8n + FMP API
RAG Q&A;	Calls /rag-ingest then /rag-query; displays GPT-4o cited answer with latency badge; supports follow-up queries	n8n RAG Workflow
Chunking Comparison	Applies 256/512/1024 chunk strategies, embeds with OpenAI, computes cosine similarity, shows bar chart	Local (OpenAI key)
Reranking Analysis	Dense-only vs BM25+semantic hybrid scoring; rank delta table, 3-component score chart	Local (Tab 3 data)

9. Setup & Deployment

9.1 Prerequisites

Requirement	Details
n8n instance	djn8n1122.app.n8n.cloud — both workflows must be Active
OpenAI API key	Required for Chunking Comparison and Reranking Analysis tabs
Python 3.10+	Streamlit app requires Python 3.10 or later
Internet access	FMP API, SEC EDGAR, OpenAI API, and n8n webhooks all require outbound HTTPS

9.2 Installation

```
cd "Gen AI/RAG_Project"

pip install -r requirements.txt

streamlit run app.py   # Opens at http://localhost:8501


# requirements.txt: streamlit>=1.35  openai>=1.30  requests>=2.31

#                   numpy>=1.26      pandas>=2.2   plotly>=5.20
```

9.3 Quick Test via curl

```
curl -X POST https://djn8n1122.app.n8n.cloud/webhook/rag-ingest \

     -H "Content-Type: application/json" -d '{"ticker":"NVDA"}'


curl -X POST https://djn8n1122.app.n8n.cloud/webhook/rag-query \

     -H "Content-Type: application/json" \

     -d '{"ticker":"NVDA","question":"What business segments does NVIDIA operate in?"
}'


curl -o NVDA_valuation.pdf "https://djn8n1122.app.n8n.cloud/webhook/stock-valuation?t
icker=NVDA"
```

10. Next Steps & Enhancements

Phase	Enhancement	Impact
Phase 1	Persistent vector store (Pinecone / Qdrant)	Survives n8n restarts; supports concurrent users
Phase 1	Full 10-K corpus from SEC EDGAR	Richer risk factors and MD&A; analysis
Phase 2	Cross-encoder reranking (ms-marco-MiniLM)	Full query-chunk attention scoring; higher precision
Phase 2	Multi-ticker comparison queries	Answer 'Compare AAPL vs MSFT' across vector stores
Phase 3	GPT-4o streaming in Streamlit	Near-instant first token; reduces perceived latency
Phase 3	RAGAS evaluation framework	Measure faithfulness, context precision, answer relevance
Phase 4	Earnings call transcript corpus	Forward-looking sentiment analysis from Q&A; transcripts

Phases 1-2: production readiness (persistence, real corpus, better reranking). Phases 3-4: UX and scope expansion (streaming, RAGAS evals, earnings transcripts).

11. Project File Reference

File	Description
app.py	Streamlit application — 4-tab financial analysis interface
requirements.txt	Python dependencies: streamlit, openai, requests, numpy, pandas, plotly
RAG_Financial_Intelligence_workflow.js	n8n SDK source code for the RAG workflow (21 nodes)
Financial_RAG_Pipeline_Deck.pptx	15-slide technical presentation with architecture diagrams
RAG_Financial_Intelligence_Documentation.pdf	This document
RAG_Project_README.md	Quick-start README with endpoint examples and architecture summary
My workflow 4-3-clean.json	WACC valuation workflow (cleaned, no personal references)

Financial RAG Intelligence Pipeline — Gen Academy Week 2 Project

`RAG Workflow: NZcW470MuDrUKjMH | Valuation Workflow: 94LnAeM8PpFn4Xob | djn8n1122.app.n8n.cloud`